

ML-Based Energy and Phase Reconstruction for the ATLAS Tile Calorimeter

Diyaa Saassouh
CERN-HSF / ATLAS
diyaa.saassouh@gmail.com

Abstract—In this report, I present a method of reconstruction of TILECAL signals using linear regression. The results, especially the Gaussian distribution of the relative residual, show the potential of this simple yet effective algorithm that can be easily run on a field-programmable gate array (FPGA). Additionally, I present a possible solution for the “bonus part”—a method to estimate the phase shift using a simple center of mass calculation. We show that the distribution of the phase shift (ps) is Gaussian and centered at 0.0375, with 68% having an absolute ps less than 0.3215, where a ps of 0 means that the target signal is perfectly timed with the digital signal, a ps of 1 means that it is timed with the next digital signal, and a ps of -1 means that it is timed with the previous digital signal.

I. INTRODUCTION

I’ll first begin by presenting how I familiarized myself with the data. Then, I’ll present how I implemented the linear regression algorithm and interpret the obtained results. I’ll attempt to reconstruct the phase and explain the results. Finally, I’ll conclude with an overview and possible continuation and changes that might further enhance the solution.

II. GETTING FAMILIAR WITH THE DATA

It is always a good idea to start by “befriending” one’s data. So, in that regard, you’ll find the `dataViewer.py` file in my submission. From this, we discover 3 keys in the file: X , y , and y_{OF} .

Since we expect to have inputs like: `samples_hi`, `samples_lo` (digital values), and `ene_hi`, `ene_lo` (target values), we quickly infer that $X[n]$ are our rows of data, $X[n][0/1]$ determines high or low gain measurements, and $X[n][0/1][0-6]$ determines the bunch crossing (BC) we’re reading from.

Similarly, $y[n]$ are the rows of target data, and $y[n][0/1]$ is the target energy of the $X[n][0/1]$ sample consisting of 7 BCs. Note that $y[n][0] \approx y[n][1]$ for all n , which makes sense since the target energy doesn’t change if the measurement gain changes.

The remaining y_{OF} seems weird at first, but a quick look at [1] reveals that y_{OF} are the values predicted by the legacy 7-coefficient linear Optimal Filtering algorithm (OF).

III. IMPLEMENTING LINEAR REGRESSION

As intended by the project description, we are going to use a linear algorithm, which in our case will be linear regression. We first load all our training data in one list, and using Scikit-Learn we implement linear regression, making sure to use the

low gain data ($X[n][1][0-6]$ and $y[n][1]$) as intended (as seen in `linearModel.py`).

IV. RESULTS AND FIGURES OF MERIT

We find that the mean of the relative residual (RR)—calculated as $(reconstructed - target)/target$ —is 0.218, in other words 21.8%, indicating a right bias in the model maybe due to phase shifts which can’t be modeled effectively linearly I think. Also a better test would probably be to take the mean of the absolute RR, however, considering the simplicity of the algorithm, it is an acceptable result.

We also get an RMS of 130, which I don’t like at all to be honest, but the reason behind it might be the low energy values that would result in a great relative error if, for example, noise and phase shift were to induce a small absolute change.

In Fig.1, we see the distribution of the RR which clearly follows a Gaussian distribution with the vast majority of the values centered around 0. In fact, if I increase the value of the bins to 200000 and plot it on a logarithmic scale (Fig. 2), we see that more than 40,000 values—almost 40% of the dataset—have an RR of less than 0.25 in absolute value and about 20000 values with RR less than 0.1.

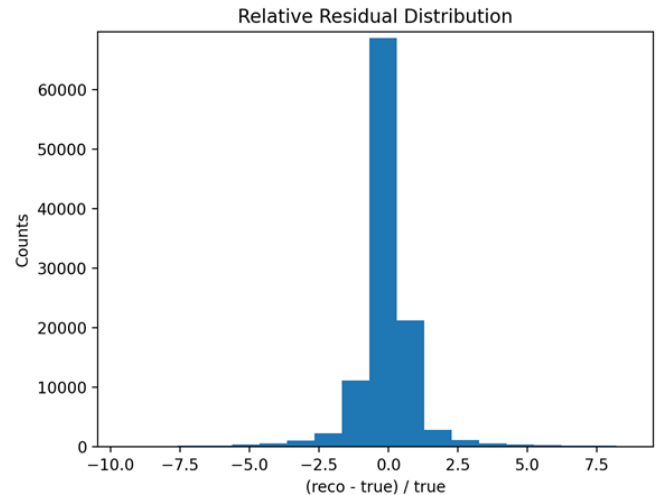


Fig. 1. Distribution of the Relative Residual, centered around 0.

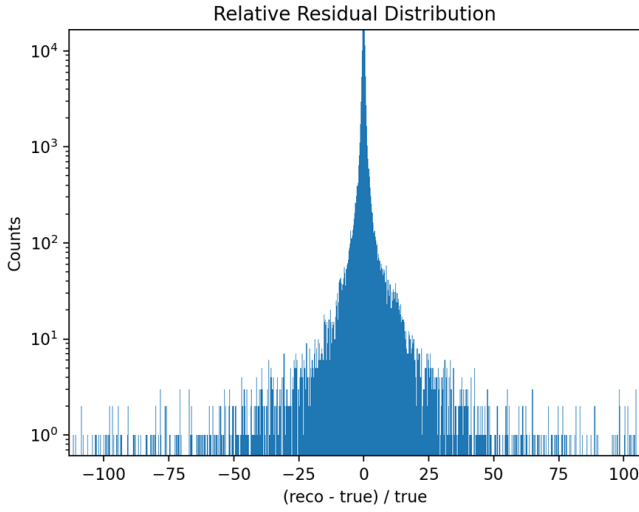


Fig. 2. Logarithmic scale of the Relative Residual distribution.

In Fig. 3, we see that the RR is close to 0 for almost all energy values greater than 0.2, but is very smeared for small energies. This result actually agrees with both the MLP and the CNN documented in [1].

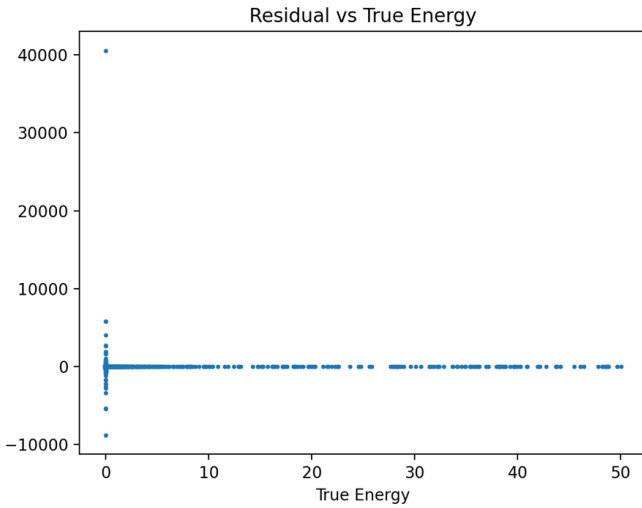


Fig. 3. 2D Distribution of the Relative Residual vs True Energy.

Additionally, I plotted the reconstructed values with respect to the true energy values (Fig. 4). We see that y_{pred} lies well on the $y = x$ line, indicating a high match with the y target. However, we also clearly notice the smear at low energy values, which partially agrees with the literature, though more complex architectures (like CNNs) have shown better results with less smearing.

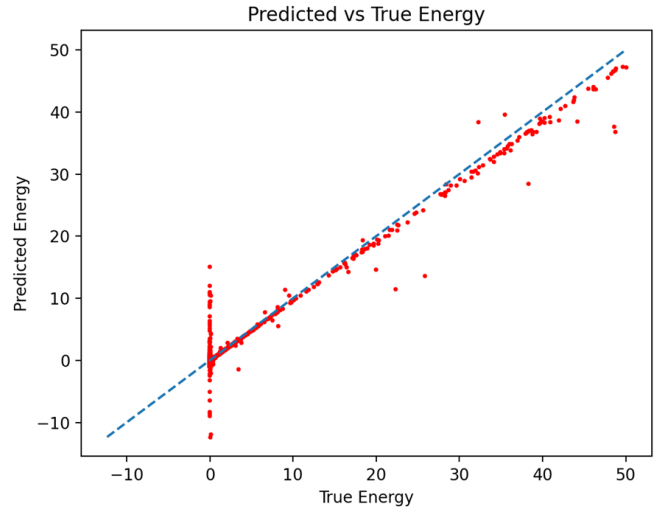


Fig. 4. Predicted Energy vs True Target Energy. The dashed line represents perfect correlation ($y = x$).

V. ATTEMPT TO RECONSTRUCT THE PHASE

The first thing that comes to mind when tackling the time shift problem is to simply check where the maximum of the 7 BCs occurs. If it occurred at a BC index less than the center, we infer a left shift; if the max occurred at an index greater than the center, we infer a right shift; and at the center, obviously no shift.

This is, however, quite trivial and will only map to discrete values from the set $\{n/7 \mid n \in [-3, 3] \cap \mathcal{Z}\}$. Following that idea, another one comes to mind which is simple yet probably more effective: calculate the “center of mass” of the pulse from the given 7 BCs, then divide the result by 3 to map it to the $[-1, 1]$ range:

$$\phi_{approx} = \frac{\sum_{i=-3}^3 iX[n][1][i+3]}{\sum_{i=-3}^3 X[n][1][i+3]} \div 3$$

where ϕ_{approx} is the approximate time or phase shift.

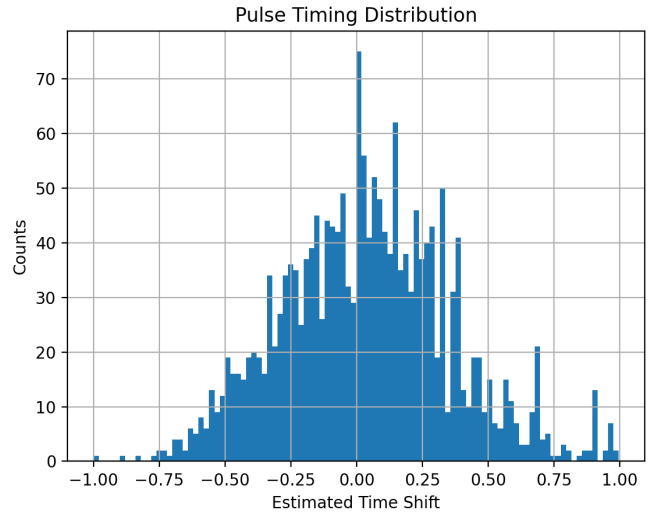


Fig. 5. Distribution of the Estimated Pulse Timing (Phase Shift).

Plotting the distribution of the time shifts (Fig. 5), we also get a Gaussian distribution with center 0.039 and 68% of values less than 0.325 in absolute value. This conforms with what is said in the hints: *The target signals are expected to be close to 0, but not exactly 0, and the time changes for each bunch crossing.*

VI. CONCLUSION

We see that although it is a very simple algorithm, its results aren't half bad. From here, one can go in two routes: either switch to more complex algorithms like the CNN or MLP, knowing fully that these aren't as easily implementable on an FPGA and aren't as fast as linear regression or Optimal Filtering; or try to modify the simple algorithm to yield better results.

For example, instead of using the 7 raw inputs to predict the target energy, we can add an eighth feature, which is actually the phase shift extracted, and see if that will improve our linear results.

REFERENCES

- [1] F. Curcio, "Machine learning-based energy reconstruction for the atlas tile calorimeter at hl-lhc," in *European Conference on Artificial Intelligence in Finance (EuCAIFCon)*. INFN Agenda, 2024, [Online]. Available: https://agenda.infn.it/event/43565/contributions/260122/attachments-136858/205493/fcurcio_EuCAIFCon.pdf.